SU Management System

Web platform for Union Officers to manage their members

G_02 - 19341848

Analysis and Specify Software Quality Requirements

Functional Requirements:

Union officers should be able to view, add, update, and remove members of their society via the web platform.

Security Requirements

Requirement Number	Security / Privacy Requirement	Rationale	Verification Method
1	The system shall require secure login using university single sign-on (SSO) and multifactor authentication for all officers	Prevent unauthorized access to society management data.	Security testing, login audit
2	The system shall encrypt all sensitive data (e.g. student IDs, emails) in transit using HTTPS and at rest using encryption	Ensure data confidentiality	Code inspection, pen testing,
4	The system shall restrict data access based on user roles (e.g. only society officers can modify member data)	Enforce least privilege access	Role-based access control testing.
5	The system shall comply with GDPR standards for data collection, retention and deletion	Protect user privacy and legal compliance	Documentation review, privacy audit

Performance Requirements
Ensure that membership management operations are processed efficiently for a smooth user experience.

Requirement Number	Performance Requirement	Rationale	Verification Method
1	The system shall load “Manage Members” page within 3 seconds under normal network conditions	Fast response improves usability	Performance testing
2	The system shall support up to 500 concurrent users performing membership management without performance degradation.	Handle peak usage (e.g. during freshers fair).	Load / stress testing
3	Updates to membership data (add/edit/remove) shall be reflected in the database within 2 seconds of submission.	Real-time data consistency.	End-to-end transaction timing.

Reliability Requirements
Ensure the system performs correctly and continuously even under the fault conditions.

Requirement Number	Reliability Requirement	Rationale	Verification Method
1	The system shall have an uptime of 99.5% during term time.	Ensure continuous availability for users.	Monitoring reports.
2	The system shall automatically back up member data every 24 hours.	Protect against data loss.	Backup verification tests.
3	The system shall recover from server failures within 5 minutes using a failover server.	Minimize downtime.	Disaster recovery testing.

4	The system shall perform input validation to prevent corrupt data entries.	Maintain data integrity	Validation and integration testing.

Scalability Requirements

Ensure the system can grow to handle increasing data and user load over time.

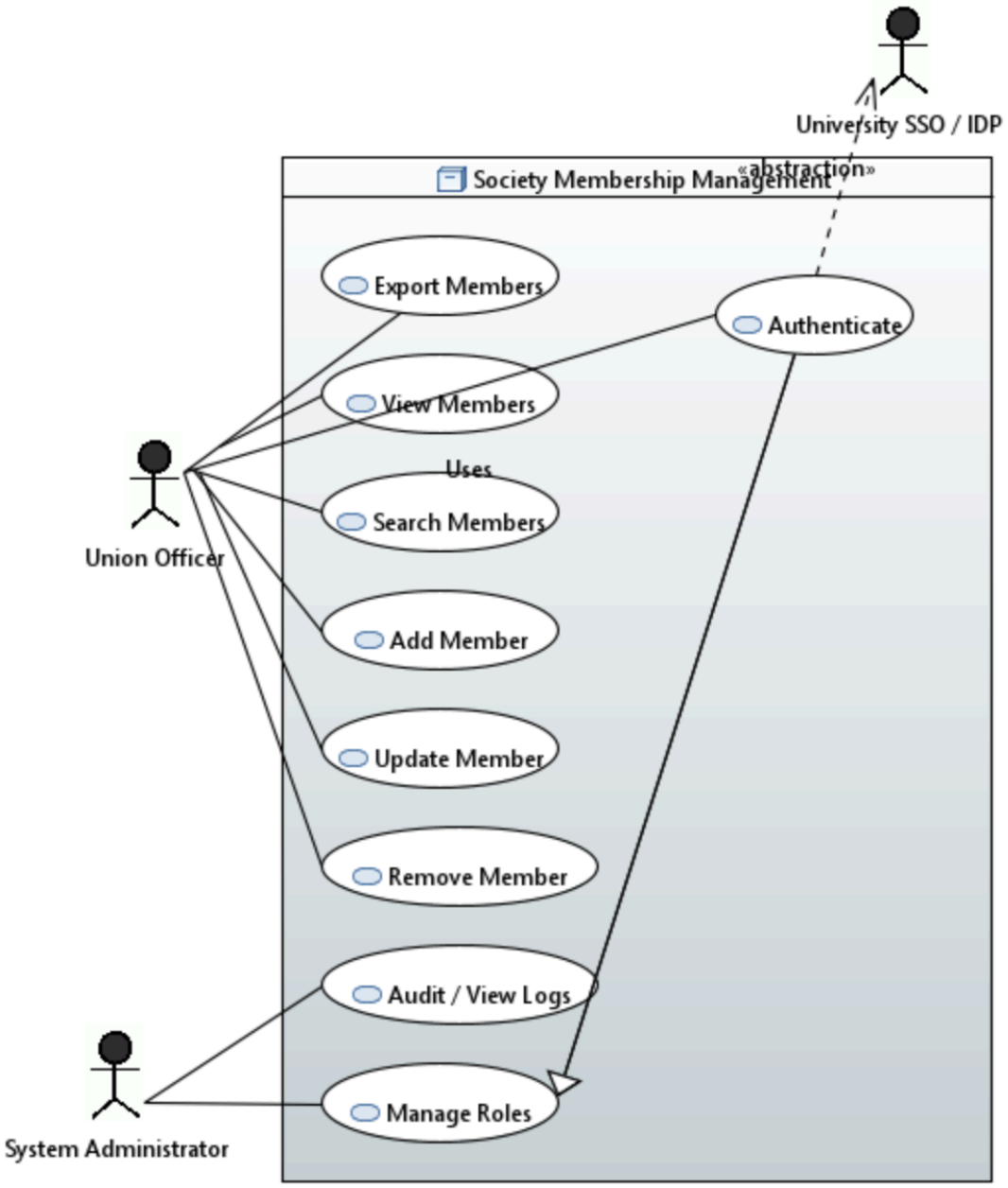
Requirement Number	Scalability Requirements	Rationale	Verification Method
1	The system architecture shall support horizontal scaling (adding more servers) without service interruption	Handle future growth in user base.	System design review.
2	The database shall efficiently handle up to 50,000 member records without performance degradation.	Support long-term data growth	Load and capacity testing
3	The system shall allow integration with new modules (e.g. event management) through RESTful apis.	Future extensibility	API integration testing.
4	The system shall use cloud-based infrastructure (e.g. AWS, azure) to dynamically allocate resources based on load.	Cost-effective scalability	Cloud environment test logs.

Summary

Quality Attribute	KeyObjectives
Security and Privacy	Protect student data using encryption, authentication, and GDPR compliance.
Performance	Fast page loads and real-time updates for membership operations
Reliability	Ensure uptime, data backups, and quick recovery from failures
Scalability	Supports increasing users, societies, and future feature integration.

User Case Diagram

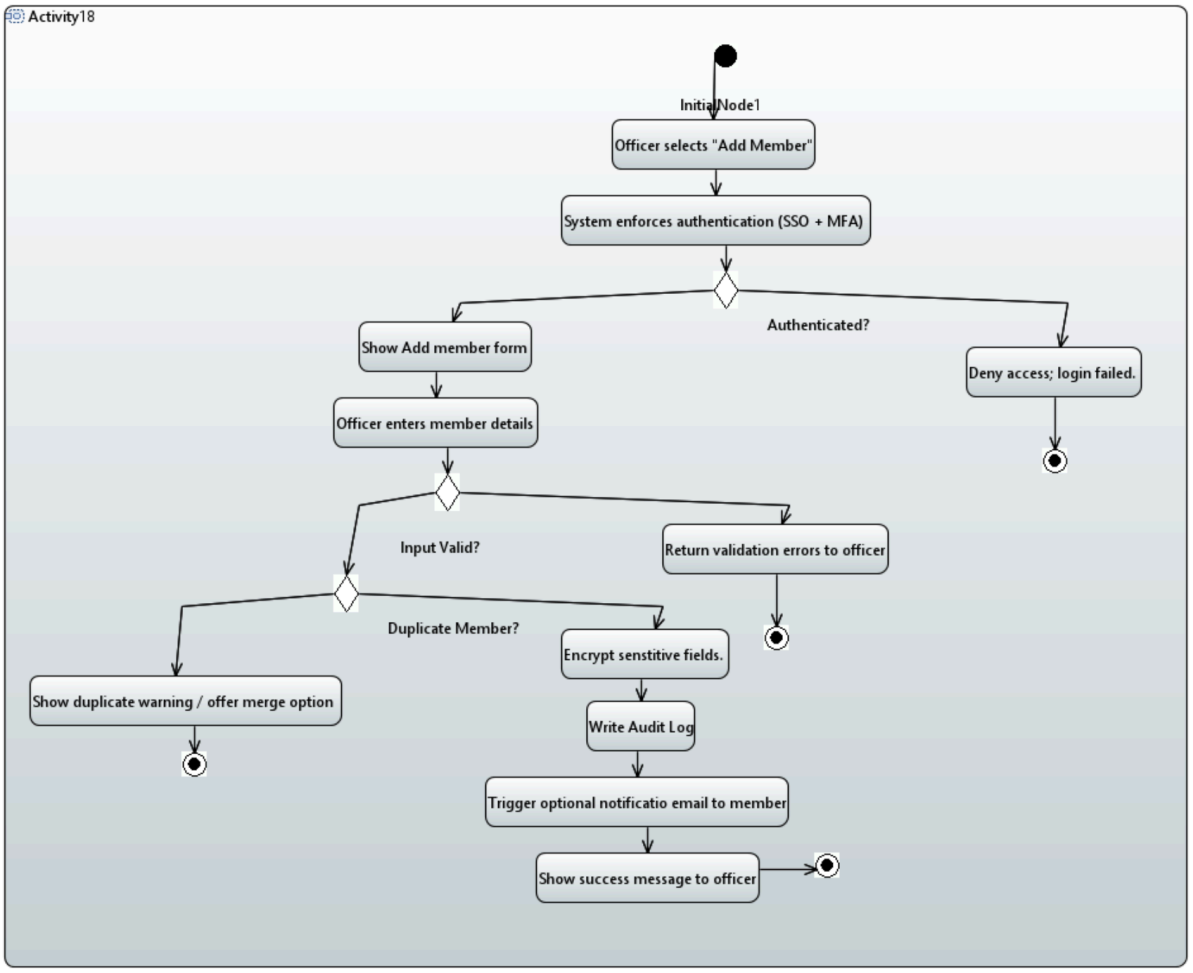
This diagram represents the user cases of the SU Membership Management section, and the interactions between officers, university SSOs and System administrators alike. Outlined within the diagram are all the functions that the union officer can complete, such as the viewing, searching, adding, updating, removing and exporting of member records. The members must first be authenticated via the SSO/ Identify provider, which the system uses to validate their identity. System administrators have enhanced capabilities, such as managing user roles and accessing audit logs to monitor system activity. All user actions rely on a shared authentication process, before accessing the system's functionalities.



Activity Diagram

The activity diagram models the workflow for adding a new member to the society membership management system. The union officer initially carries out this by selecting the “Add Member” section. This then boots the system to begin a secure authentication process through SSO (single sign on) and MFA (multifactor authentication). If the authentication of this process fails, the system then denies access and ends the process of adding the member. However if this process succeeds the officer is then prompted to enter the new member’s details into the system.

After the officer has entered the new member's details, the system validates the input through a set of algorithms. Invalid data will result in an output from the system and an end the activity, where the union officer can begin the process again. If the input was valid, however, the new records are checked across the database to ensure that the member is not a duplicate. If they are duplicated, the process starts over, the new member is not added. If no duplicates are found within the database, a complex encryption algorithm is completed to sensitive fields, so the data can be securely stored. Next the audit log of the member being added is saved, to allow for GDPR and traceability compliance. An optional function is to send a message to the user to confirm that their account has been successfully added. Finally, the system displays a success message to the union officer, confirming that the member was successfully created.

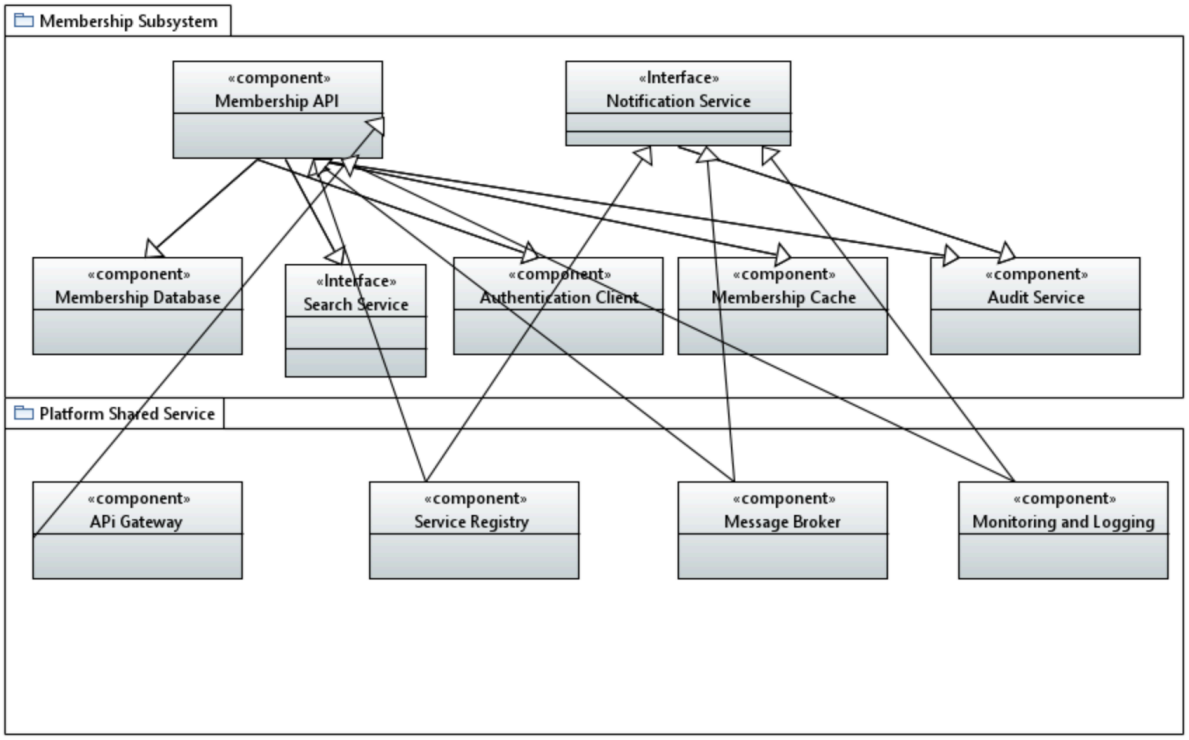


Component Diagram

Although it is a bit messy (limitations of papyrus) the component diagram illustrates the overall software architecture of the subsystem I was tasked with designing. It is separated into two logical groups, the membership system and the platform shared services. The membership subsystem contains the core devices, including the Membership API and database. These components together, implement the core functionality around managing membership data, authentication, caching, searching, and auditing.

The Membership API, is shown as the central orchestrating component, interacting with nearly all other components within the subsystem as a whole. It communicates with the databases to perform operation, and utilises caching for performance optimization (as per quality requirements). It also invokes the authentication client for identity verification and integration with the search services for indexing and queries. It also interacts with the audit service to record the events, and communicates with the notification services for sending out key notifications to the members.

Finally, the lower tiers are for platform shared services, this represents the supporting infrastructure used by the membership subsystem. These include the API gateway, service registry, message broker and Monitoring and Logging services. The membership API and other subsystem components heavily rely on these services for concerns such as discovery, centralized routing, event messaging, and operational monitoring.

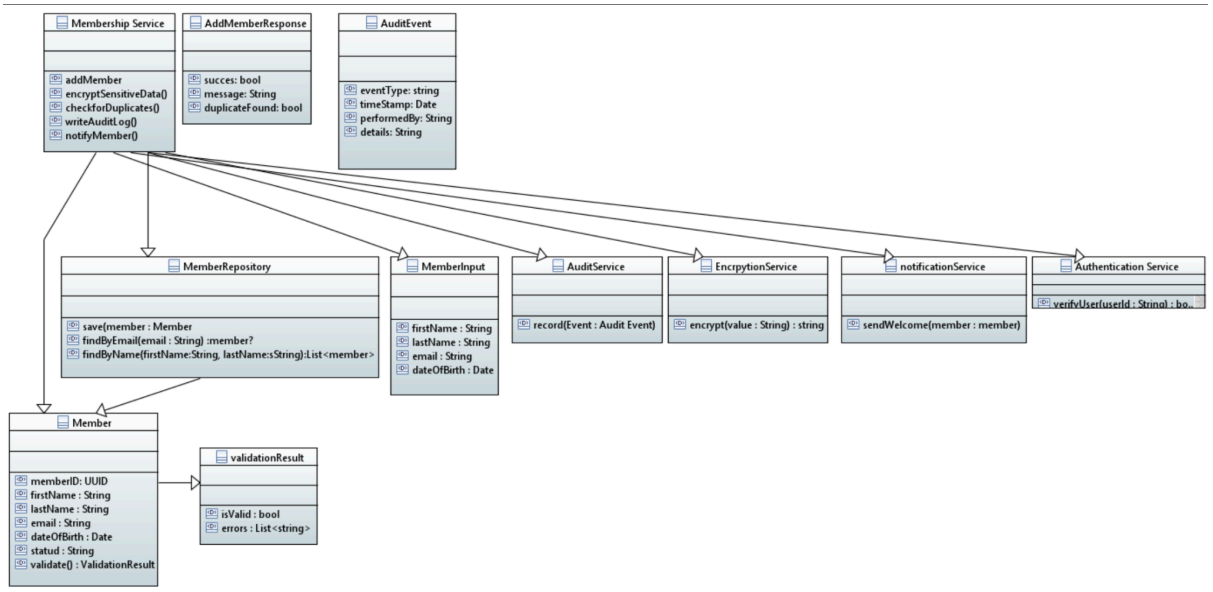


Class Diagram - Add Members

This diagram outlines the service for adding new members to the database. The central class MembershipService, controls the flows through the methods outlined. When a member is added, the service memberResponse, which communicates whether the operation is successful, as per the activity diagram outlined above. It also questions whether there was a duplicate found when adding the member.

Data about new users flows in through the MemberInput class, which contains fields such as the names of the new members, DOB and their email. This input is moved into the member object, added an ID, status and validation logic. Results are encapsulated in the validationResult class. The memberRepository class, allows for officers to look up members by email or name when checking for duplicates.

The external services support the membership service through services such as encryption and auditing actions that have been completed on the database. Welcome messaging service also sends a message to the new member, as a way of confirming that their account has been created.



Team Work Performance

Overall the team worked successfully in order to achieve the goal, meeting frequently in order to set out objectives for the coming week. Attendance was generally good, from myself and 19324102. However, if we were to complete this module again, I would have liked us to be more proactive from the start. This would allow for easier coursework in the long run. Unfortunately, one of our team mates was very absent right up until the later stages. This meant that we needed to catch him up on what he had missed out on. I'd say this delayed the project slightly longer than I would have liked it to. On top of this while 19310578 was present at the start, towards the middle and latter stages of the development of this project, attendance to meetings tailed off, and responding to the group chat also began to become sparse. We didn't see what progress of work she had made for a very long time, after their initial commits.
The structure of the group allowed for an adaptive working environment. The module leader (19324102) operated successfully in organizing the github file structure. I operated as a 2nd in command (or a 2IC) frequently taking meeting notes, and setting up the communication platform and github initially. I was able to take my experience being in the military (Reserves) to effectively carry out a "chain of command" structure to the group. This allowed for greater efficiency in the project, especially key due to the ever looming time constraint.
Despite the challenges, particularly concerning attendance, the team was able to rally together and complete this project within the deadline set. Shared responsibility grew stronger during the final weeks, really pushing everyone to put in the work required to complete the project.
As a personal takeaway, which I can take forward into future projects, I will strive to lay down the importance of accountability within the team-base project, and make clear what I expect from both myself and my fellow team members. University group projects can always be challenging and therefore adaptability and proactiveness are required in order to carry out projects to a high standard.

Conclusion

Planning a development of a software system such as the SU Membership Management has demonstrated how vital it is for a clearly defined software quality requirements and well structured system design before even thinking about implementing the code. By clearly outlining the requirements in domains such as security, performance, reliability and scalability, the project ensures that the proposed system would not only meet the expectation, but also remain robust, compliant and most importantly future proof. The diagrams that I have created using the software Papyrus give a basic visual representation of how I would go about applying the development of this piece of software. It also allows for clear communication and collaboration between my group members.
The project also highlighted the value of effective teamwork and adaptive collaboration in achieving a complex software engineering task such as this. Although some students were not consistent in attendance and ensuring work was completed to our internal deadlines, they were able to show their commitment to completing the project, although a little late…
Overall this project was successful in meeting all of its requirements, and it has been able to offer a good trial run on working a large scale project within a group.